AI1013_assignment_5_AI24BTECH11015_2

May 1, 2025

1 AI1013 : Programming for AI

1.1 Assignment 5

1.1.1 2. Wordle: A Game in Probability

2.1 Dataset Importing required libraries.

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import cvxpy as cp
```

Reading the CSV

[2]:
```
wordlist = pd.read_csv("wordlist.csv", header=None)
```

2.2 Data Exploration and Setup Verifying if the wordlist was loaded correctly

[3]:
```
N = len(wordlist)
print(N)
```

```
3103
```

Great! Now lets compute and visualize the histogram as directed.

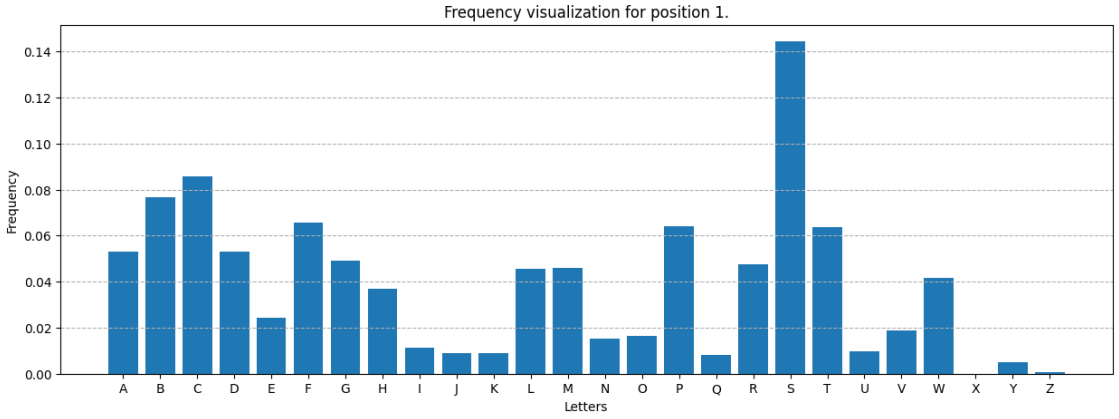
Starting by defining a function that calculates and plots the required histogram.

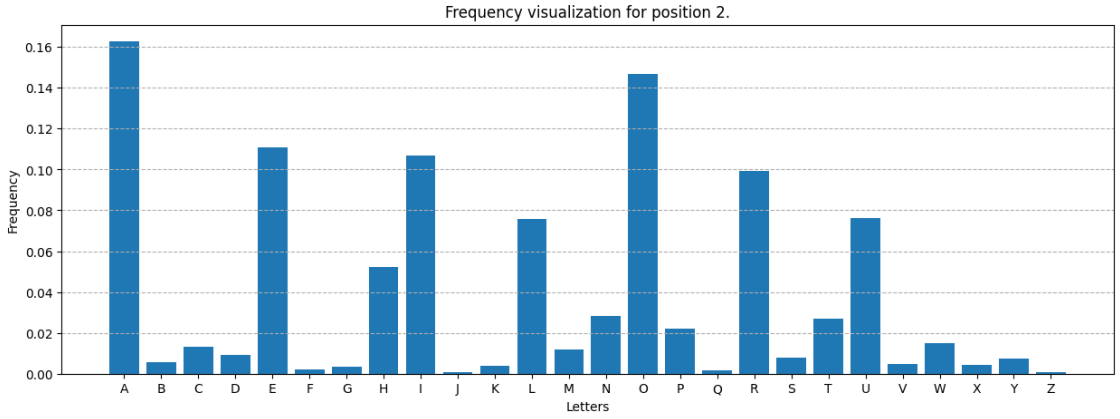
[4]:
```
def histogram_visualization(wordlist, position):
    #for 0 based indexing
    index = position - 1

    #initializing empty frequency array
    frequency ={chr(i): 0 for i in range(ord('A'), ord('Z') +1)}


    #getting the frequency of the letters at the given position
    for word in wordlist[0]:
        frequency[word[index]] = frequency.get(word[index],0 ) + 1

    #we will be plotting alphabetically
```

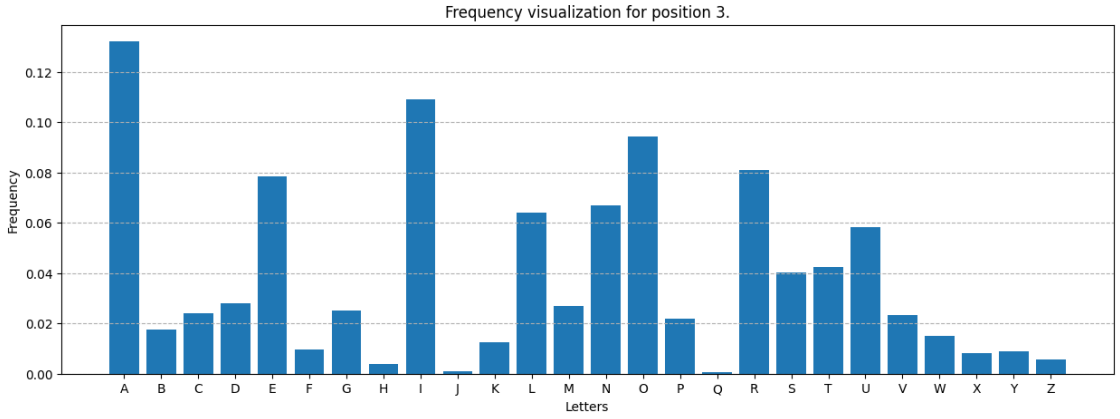
```
    sorted_frequency = sorted(frequency.items())

    #preparing for plotting
    size = N
    letters = [freq[0] for freq in sorted_frequency]
    counts = [freq[1]/size for freq in sorted_frequency]


    #plotting
    plt.figure(figsize=(15,5))
    plt.bar(letters, counts)
    plt.title(f"Frequency visualization for position {position}.")
    plt.xlabel("Letters")
    plt.ylabel("Frequency")
    plt.grid(axis="y", linestyle="--")
    plt.show()
```

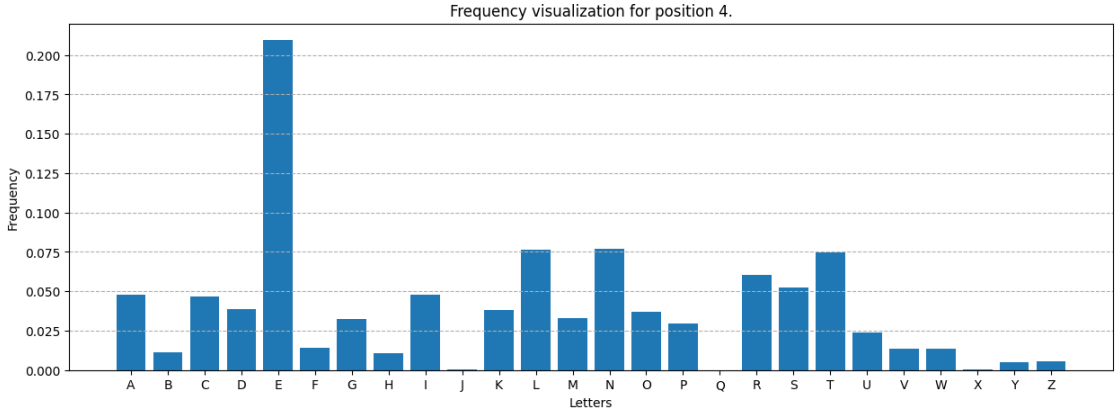
Calling it and plotting the histograms

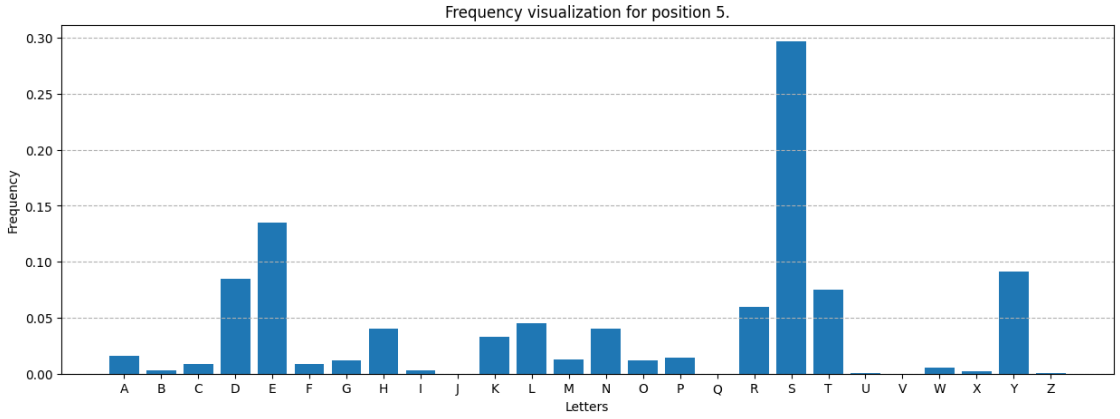
[5]:
```
#plotting usin the above function
for position in range(1,6):
    histogram_visualization(wordlist, position)
```




Frequency visualization for position 2.
Frequency
0.16
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Letters


Frequency visualization for position 3.
Frequency
0.12
0.10
0.08
0.06
0.04
0.02
0.00
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Letters


Frequency visualization for position 4.
Frequency
0.200
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Letters



The plots are here. We observe that

1. For position 1, letter 'S' dominates, while majority of letters appearing with frequency in similar range. Some of the letters are very rare.
2. For position 2, only a few letters dominate the graph, others being rare.
3. For positon 3, again the distribution is somewhat like for position 1, but with no clear dominator this time, and more letters that occur rarely.
4. For position 4, the frequency distribution is almost same as that for position 1.
5. For position 5, only a few letters are probable to appearm with again the letter 'S' dominating the graph.

2.3 Simulating Wordle Feedback Lets define the `wordle_feedback` function as instructed

[6]:
```
def wordle_feedback(guess, solution):
    #list to track yellow tiles (or gray)
    letters_incorrect_pos = []

    #count for tiles
    green_tiles = 0
    yellow_tiles = 0

    #unused indices in solution:
    unused_indices = []


    #first will check for greens if correct, increment one
    for i in range(5):
        if guess[i] == solution[i]:
            green_tiles +=1
        else:
            unused_indices.append(i)
```

```
            letters_incorrect_pos.append((i,guess[i]))

    #to track used solution indices
    used_solution_indices = []

    #now we'll check for the yellow tiles
    for i, letter in letters_incorrect_pos:
        for j in unused_indices:
            if j not in used_solution_indices and letter == solution[j]:
                yellow_tiles +=1
                used_solution_indices.append(j)
                break

    #remaining are gray tiles
    gray_tiles = 5-green_tiles - yellow_tiles

    return (green_tiles, yellow_tiles, gray_tiles)
```

Lets check if this function works fine for the given words

[7]:
```
print(wordle_feedback("CRANE", "SLATE"))
#expected (2,0,3)
```

```
(2, 0, 3)
```

[8]:
```
print(wordle_feedback("BRAIN", "BRINE"))
#expected (2,2,1)
```

```
(2, 2, 1)
```

Our function seems to be working fine, Lets proceed further.

2.4 Probability Estimation via Averages

1. Estimating the probability that a guess word results in atleast one green tile

[9]:
```
np.random.seed(42) #for reproducability


#fixed guess word
guess_word = "CRANE"

def calculate_words_with_green_tile_prob(wordlist, guess_word,␣
 ↪num_solution_words):
    #counting variable
    words_with_green_tiles = 0

    #selecting words randomly
    indices = np.random.choice(len(wordlist), num_solution_words)
```

```
    for index in indices:
        green, yellow, gray = wordle_feedback(guess_word, wordlist[0][index])
        if green:
            words_with_green_tiles +=1

    return (words_with_green_tiles/num_solution_words)*100


#calculating probability
print("The probability that the word 'CRANE' results in at least one green tile␣
 ↪= ", calculate_words_with_green_tile_prob(wordlist, guess_word, 1000), "%")
```

```
The probability that the word 'CRANE' results in at least one green tile =  37.8
%
```

2. Estimating the probability that a guess word results in exactly two yellow tiles

[10]:
```
#fixed guess word
guess_word = "CRANE"

def calculate_words_exactly_two_yellow_prob(wordlist, guess_word,␣
 ↪num_solution_words):
    #to count the 1/0
    words_with_two_yellow = 0

    #selecting words randomly
    indices = np.random.choice(len(wordlist), num_solution_words)

    for index in indices:
        green, yellow, gray = wordle_feedback(guess_word, wordlist[0][index])
        if yellow==2:
            words_with_two_yellow +=1

    return (words_with_two_yellow/num_solution_words)*100

#calculating probability
print("The probability that the word 'CRANE' results exactly two yellow tiles =␣
 ↪", calculate_words_exactly_two_yellow_prob(wordlist, guess_word, 1000), "%")
```

```
The probability that the word 'CRANE' results exactly two yellow tiles =  24.4 %
```

2.5 A Greedy Strategy and its Average Guessing Time Lets define a function to implement the greedy strategy.

[11]:
```
def check_feedback(word, solution_word, feedback):
    #not including the current solution_word as we already know this is not the␣
 ↪answer
    #hence reducing the search space by one more word at every step
    if word == solution_word:
```

```
        return False

    curr_feedback = wordle_feedback( solution_word, word)

    #checking if the feedback of this word with our greedy guess is the same as␣
↪feed back of the greedy guess witht the fixed guess word or no
    result = curr_feedback == feedback
    return result


def greedy_strategy(wordlist, guess_word, num_trials):
    num_steps_required = []

    for i in range(num_trials):
        #initial filtered words is the complete wordlist
        filtered_words = wordlist[0].copy()
        steps = 0

        #loop goes on till stopped by condition
        while True:
            #randomly picking a solution word
            solution_word = np.random.choice(filtered_words)

            steps +=1

            #getting feedback for the current solution
            current_feedback = wordle_feedback(solution_word, guess_word)

            #breaking if our solution word is the fixed guess word
            if current_feedback[0] == 5:
                break

            #filtering to reduce the wordlist size
            selected_indices = filtered_words.apply(check_feedback,␣
↪args=(solution_word, current_feedback))
            filtered_words = filtered_words[selected_indices]

        num_steps_required.append(steps)

    return np.mean(num_steps_required)
```

Running for “VERVE” and “CRANE” as directed

[12]:
```
avg_steps_verve = greedy_strategy(wordlist, "VERVE", 1000)
avg_steps_crane = greedy_strategy(wordlist, "CRANE", 1000)

print("For 'VERVE', avg steps = ", avg_steps_verve)
```

```
print("For 'CRANE', avg steps = ", avg_steps_crane)
```

```
For 'VERVE', avg steps =  5.51
For 'CRANE', avg steps =  5.426
```

So, this greedy method using the `wordle_feedback` function defined earlier solves the wordle in 5.5 steps on average.

2.6 Evaluating Different Strategies for choice of guess Looking for a word that satisfies S_1's proposal.

[13]:
```
#to store the probabilites of each word
probabilities = []

for word in wordlist[0]:
    current_probability = calculate_words_with_green_tile_prob(wordlist, word, N)
    probabilities.append((current_probability, word))

#getting the maximum probability and storing all words with this probability in␣
 ↪possible_words
max_probability = max(probabilities, key = lambda x: x[0])[0]
possible_words = [word for prob, word in probabilities if prob ==␣
 ↪max_probability]

#randomly choosing a word
word_s1 = np.random.choice(possible_words)

print(f"According to S1, we should choose ", word_s1, "which has the␣
 ↪probabaility of getting atleast one green tile = ",␣
 ↪calculate_words_with_green_tile_prob(wordlist, word_s1, N))
```

```
According to S1, we should choose  SALES which has the probabaility of getting
atleast one green tile =  62.93909120206253
```

Let's now look for word that satisfies S_2's proposal

[14]:
```
#same as above
probabilities = []

for word in wordlist[0]:
    current_probability = calculate_words_exactly_two_yellow_prob(wordlist,␣
 ↪word, N)
    probabilities.append((current_probability, word))

max_probability = max(probabilities, key = lambda x: x[0])[0]
possible_words = [word for prob, word in probabilities if prob ==␣
 ↪max_probability]

#randomly choosing a word
```

```
word_s2 = np.random.choice(possible_words)

print("According to S2, we should choose ", word_s2, "which has the probabaility␣
↪of getting exactly two yellow tiles = ",␣
↪calculate_words_exactly_two_yellow_prob(wordlist, word_s2, N))
```

```
According to S2, we should choose  AISLE which has the probabaility of getting
exactly two yellow tiles =  35.12729616500161
```

for finding according to S_3, defining a function to calculate probabiltiy of atleast two yellow tiles.

[15]:
```
def calculate_words_with_atleast_2yellow(wordlist, guess_word,␣
↪num_solution_words):
    #to count
    words_with_atleast_2yellow = 0

    #selecting words randomly
    indices = np.random.choice(len(wordlist), num_solution_words)

    for index in indices:
        green, yellow, gray = wordle_feedback(guess_word, wordlist[0][index])
        if yellow>=2:
            words_with_atleast_2yellow +=1

    return (words_with_atleast_2yellow/num_solution_words)*100
```

Now finding according to S_3

Change here for using exactly2 or atleast 2 yellow tiles

[16]:
```
#we will first precompute the g and y values for all the words in the wordlist
#note that we are using the complete wordlist to calculate the probability
word_probabilities ={}
for word in wordlist[0]:
    g = calculate_words_with_green_tile_prob(wordlist, word, N)
    y = calculate_words_with_atleast_2yellow(wordlist, word, N)
    word_probabilities[word] = (g,y)

words = list(word_probabilities.keys())
g_list = [word_probabilities[w][0] for w in words]
y_list = [word_probabilities[w][1] for w in words]
```

Lets solve it using cvxpy

[17]:
```
#defining the variables
alpha = cp.Variable()
F = cp.Variable()

#constraints
```

```
constraints = [alpha>=0, alpha<=1]

for g,y in zip(g_list, y_list):
    constraints.append(F <= alpha*g + (1-alpha)*y)

#formulating the problem
problem = cp.Problem(cp.Maximize(F), constraints)

#solving
problem.solve()

#retrieveing the optimal values
optimal_alpha = alpha.value
optimal_F = F.value.item()
print(f"optimal alpha  = {optimal_alpha}")
print(f"optimal F = {optimal_F}")
```

```
optimal alpha  = 0.977064220171003
optimal F = 10.476100370428057
```

Now that we have the optimal value of F and α, lets look for the word that gets this optimal value

[18]:
```
#we will look for the word which gives minimum difference with the optimal value␣
 ↪of F
min_diff = float('inf')
best_word = None

for word, (g,y) in word_probabilities.items():
    value = optimal_alpha*g +(1-optimal_alpha)*y

    diff = abs(value-optimal_F)
    if diff<min_diff:
        min_diff = diff
        best_word = word
        min_value = value


print(f"According to S3,we should choose the word : {best_word}")
print(f"Value of F for {best_word} comes out to be {min_value:.4f}")
```

```
According to S3,we should choose the word : AFFIX
Value of F for AFFIX comes out to be 10.4761
```